Project Report

เรื่อง Light Out Puzzle

เสนอ

รศ.ดร.รังสิพรรณ มฤคทัต

จัดทำโดย

นางสาวกรชนก ภู่ตระกูล 6713115

นางสาวณัฏฐา ลิ้มคุณธรรมโม 6713117

นายจักริน เริ่มตั้งสกุล 6713221

รายงานนี้เป็นส่วนหนึ่งของวิชา Structure and Algorithm

ภาคเรียนที่ 2 ปีการศึกษา 2568

คำนำ

รายงานฉบับนี้เป็นส่วนหนึ่งของวิชา Data Structure and Algorithm (EGCO 221) จัดทำขึ้นเพื่อใช้ประกอบอธิบายการทำงานโปรแกรมของ Light Out Puzzle ประกอบไปด้วยส่วนของคู่มือการใช้งานโปรแกรมเบื้องต้น และการอธิบายในส่วนของ Code/Algorithm รวมไปถึงข้อจำกัดต่างๆ ในการใช้งานโปรแกรม

ทางคณะผู้จัดทำขอขอบพระคุณ รศ.ดร.รังสิพรรณ มฤคทัต ผู้ให้ความรู้ และแนวทางการศึกษา

สุดท้ายนี้ทางคณะผู้จัดทำหวังว่ารายงานฉบับนี้จะสามารถเป็นประโยชน์ไม่มากก็น้อยแก่ผู้อ่านทุกท่าน

ขอขอบพระคุณ

คณะผู้จัดทำ

สารบัญ

1. คู่มือการใช้งานโปรแกรม (Short User Manual) **4**

1.1 การเริ่มต้นโปรแกรมและกำหนดขนาด Grid 4

1.2 การกำหนดสถานะเริ่มต้นของหลอดไฟ 4

1.3 การกำหนดหลอดไฟเสีย 4

1.4 ดูผลลัพธ์การค้นหา 5

1.5 การเล่นซ้ำหรือออกจากโปรแกรม 5

2. โครงสร้างข้อมูล (Data Structures) **7**

2.1 โครงสร้างข้อมูลกราฟ (Graph Data Structure) 7

2.2 โครงสร้างข้อมูลอื่นๆ 10

3. อัลกอริทึมกราฟ BFS ในการดับไฟ **11**

3.1 อัลกอริทึมที่ใช้: BFS (Breadth-First Search) 11

3.2 ขั้นตอนการค้นหา (Forwarding Steps) 11

3.3 กรณีไม่มีคำตอบ (No Solution) 12

4. การสาธิตการทำงานของอัลกอริทึม (Demonstration) **14**

4.1 Demo 1: Grid 2×2, ไม่มีไฟเสีย, initial state = 1111 14

4.2 Demo 2: Grid 2×2, มีไฟเสียที่ (0,0), initial state = 1111 16

5. ข้อจำกัดของโปรแกรม (Program Limitation) **19**

5.1 ข้อจำกัดด้านหน่วยความจำและเวลา (BFS State Space) 19

5.2 ไฟเสียรองรับได้สูงสุด 1 ดวง 19

5.3 การ copy history ทุก state ทำให้ใช้ memory มากกว่าจำเป็น 20

1. คู่มือการใช้งานโปรแกรม (Short User Manual)

1.1 การเริ่มต้นโปรแกรมและกำหนดขนาด Grid

เมื่อเริ่มรันโปรแกรม จะปรากฏข้อความแสดง prompt รอรับขนาด Grid N×N โดยค่า N ต้องเป็นจำนวนเต็มที่มีค่าอย่างน้อย 2

```
Enter grid size N (N x N, min 2):2
```

- หากป้อนค่าที่ไม่ใช่จำนวนเต็ม เช่น 0.5 โปรแกรมจะแจ้งเตือนว่า [!] Please enter a valid integer.
- หากป้อนค่าน้อยกว่า 2 โปรแกรมจะแจ้งเตือนว่า [!] N must be at least 2.
- โปรแกรมจะถามซ้ำจนกว่าจะได้ค่าที่ถูกต้อง

1.2 การกำหนดสถานะเริ่มต้นของหลอดไฟ

โปรแกรมจะถามให้ป้อน bit string ขนาด N×N โดยใช้ตัวเลข 1 (ไฟติด) และ 0 (ไฟดับ) เรียงจากซ้ายไปขวา บนลงล่าง

```
Enter initial state (4 bits, left to right, e.g. 1001):1111
```

- ต้องมีความยาวเท่ากับ N×N ตัวอักษรพอดี และใช้เฉพาะ '0' หรือ '1' เท่านั้น
- หากป้อนไม่ถูกต้อง โปรแกรมจะแจ้งเตือนและถามใหม่

จากนั้นโปรแกรมจะแสดงสถานะกระดานเริ่มต้น เช่น

```
      | col 0| col 1|
row 0 |  1   |  1   |
row 1 |  1   |  1   |
```

1.3 การกำหนดหลอดไฟเสีย

โปรแกรมจะถามว่าต้องการกำหนดหลอดไฟเสียหรือไม่

```
Set broken light (Y/N)?y
Enter row of broken light (0-1) =0
Enter col of broken light (0-1) =0
```

- กด Y เพื่อกำหนดตำแหน่งหลอดไฟเสีย แล้วกรอก row และ col
- กด N หากไม่มีหลอดไฟเสีย
- หลอดไฟเสียจะแสดงด้วยเครื่องหมาย 'x' ต่อท้ายตัวเลขบนกระดาน เช่น 1x

- โปรแกรมจะ validate ว่า row และ col อยู่ในช่วง 0 ถึง N-1 หากไม่ถูกต้องจะถามใหม่

1.4 ดูผลลัพธ์การค้นหา

โปรแกรมจะค้นหาวิธีดับไฟทุกดวงโดยอัตโนมัติด้วย BFS และแสดงผลทีละ move

```
Start searching for solution (BFS)...
3 move(s) to turn off all lights
>> Move 1 : toggle row 0, col 0
   State in bits = 0110
      | col 0| col 1|
row 0 |  0x  |  1   |
row 1 |  1   |  0   |
>> Move 2 : toggle row 0, col 1
   State in bits = 1011
      | col 0| col 1|
row 0 |  1x  |  0   |
row 1 |  1   |  1   |
>> Move 3 : toggle row 1, col 0
   State in bits = 0000
      | col 0| col 1|
row 0 |  0x  |  0   |
row 1 |  0   |  0   |
```

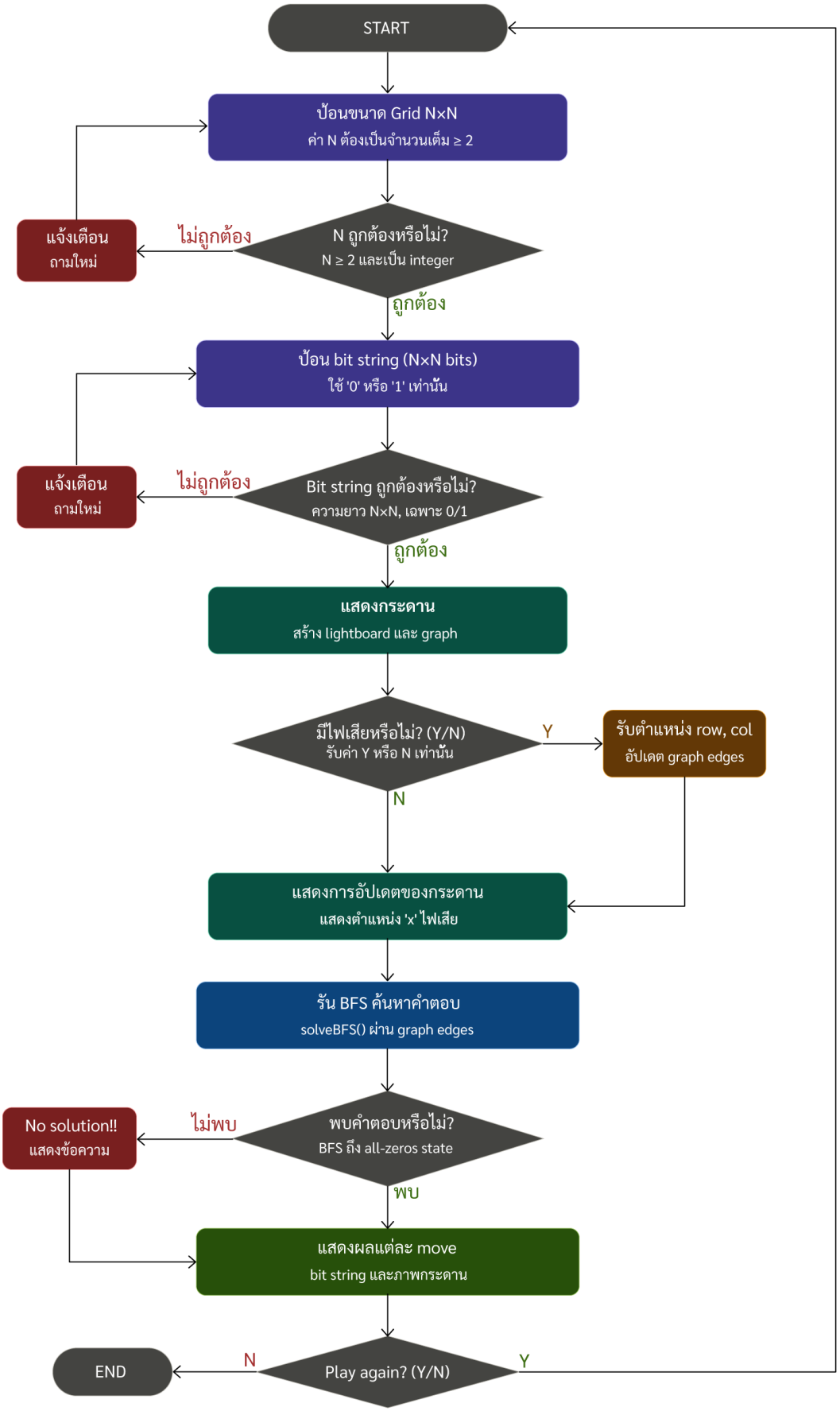
- แต่ละ move จะแสดงพิกัดที่กด สถานะ bit string และภาพกระดาน
- หากไม่มีทางดับไฟได้ โปรแกรมจะแสดงข้อความ No solution!!

1.5 การเล่นซ้ำหรือออกจากโปรแกรม

หลังจบการค้นหา โปรแกรมจะถามว่าต้องการเล่นใหม่หรือไม่

```
Play again? (Y/N): y
```

- กด Y เพื่อเริ่มเกมใหม่ด้วย N และ initial state ใหม่
- กด N เพื่อออกจากโปรแกรม



ภาพที่ 1-1: Flowchart การใช้งานโปรแกรม

2. โครงสร้างข้อมูล (Data Structures)

2.1 โครงสร้างข้อมูลกราฟ (Graph Data Structure)

2.1.1 ตำแหน่งของกราฟในโปรแกรม

กราฟถูกสร้างและใช้งานในคลาส lightboard โดย object ที่เก็บกราฟมีชื่อว่า overboard ซึ่งประกาศเป็น field ของคลาส ดังนี้

```
class lightboard{
    private DefaultDirectedGraph<light, DefaultEdge> overboard ;
    private light[][] node;
    private int board;
    public lightboard(int size,String initial){
        board = size;
        overboard =  new DefaultDirectedGraph<>(DefaultEdge.class);
        node = new light[size][size];
        for(int i=0;i<board;i++)
            for(int j=0;j<board;j++)
            {
                boolean isOn = initial.charAt(i * board + j) == '1';
                node[i][j] = new light(i,j,isOn);
                overboard.addVertex(node[i][j]);
            }
        for (int r = 0; r < board; r++) {
            for (int c = 0; c < board; c++) {
                createEdgesForNode(node[r][c]);
            }
        }
    }
```

กราฟถูกสร้างใน constructor ของ lightboard ถูกอัปเดตใน method setBroken() เมื่อมีการกำหนดหลอดไฟเสีย และถูกใช้ใน solveBFS() ผ่าน simulatePress() เพื่อดึง edge สำหรับจำลองการ toggle

2.1.2 ความหมายของ Node และ Edge

องค์ประกอบ	ความหมาย
Node (โหนด)	แทนหลอดไฟแต่ละดวงใน Grid โดยแต่ละ node เก็บข้อมูล: ตำแหน่ง (row, col), สถานะ on/off และสถานะ broken
Edge (เส้นเชื่อม)	แทนความสัมพันธ์ของการ toggle: ถ้ามี directed edge จาก node A ไปยัง node B หมายความว่าเมื่อหลอดไฟ A ถูกกด หลอดไฟ B จะถูก toggle ตามไปด้วย

ตารางที่ 2-1: ความหมายของ Node และ Edge ในกราฟ

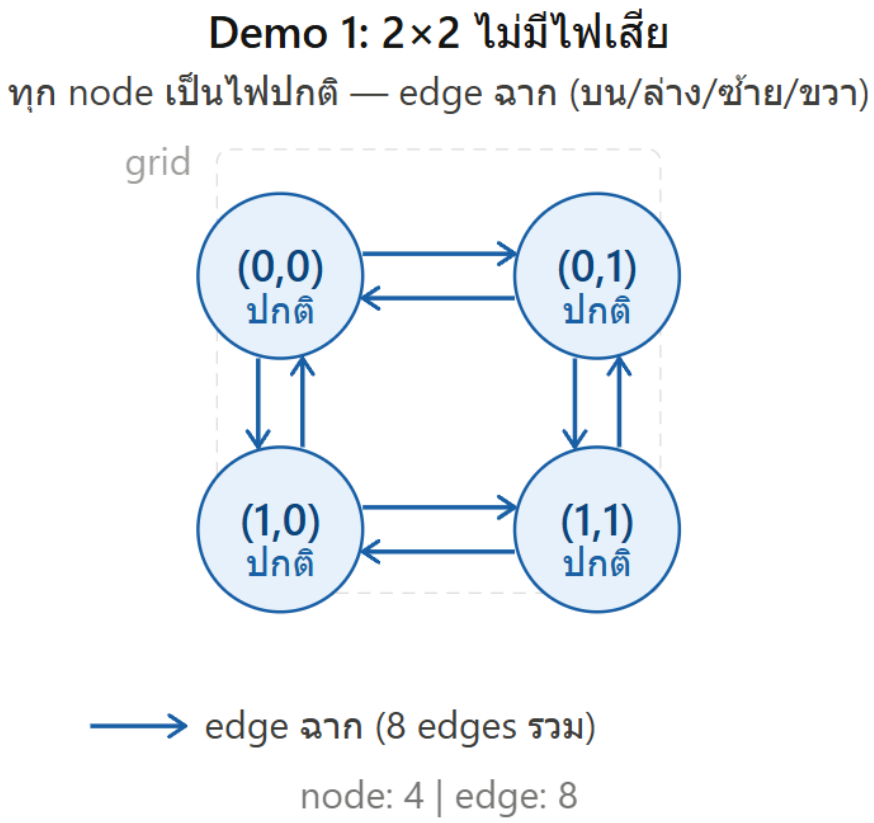
2.1.3 เงื่อนไขการสร้าง Edge (method createEdgesForNode)

```java
    private void createEdgesForNode(light node) {
        int r = node.getRow();
        int c = node.getCol();
        if (!node.getbroken()) {
            int[][] neighbors = {{r-1, c}, {r+1, c}, {r, c-1}, {r, c+1}};
            addValidEdges(node, neighbors);
        } else {
            int[][] diagonals = {{r-1, c-1}, {r-1, c+1}, {r+1, c-1}, {r+1, c+1}};
            addValidEdges(node, diagonals);
        }
    }
```

- หลอดไฟปกติ (broken = false): สร้าง edge ไปยังเพื่อนบ้านบน ล่าง ซ้าย ขวา ที่อยู่ภายใน Grid
- หลอดไฟเสีย (broken = true): สร้าง edge ไปยังเพื่อนบ้านแนวทแยง 4 ทิศ ที่อยู่ภายใน Grid
- method addValidEdges() ทำหน้าที่กรอง node ปลายทางว่าอยู่ใน Grid ก่อนเพิ่ม edge

2.1.4 กราฟจริงของ Demo 1 (2×2 ไม่มีไฟเสีย, initial = 1111)

กระดาน 2×2 มี node ทั้งหมด 4 node คือ (0,0), (0,1), (1,0), (1,1) ทุก node เป็นหลอดไฟปกติ จึงสร้าง edge ไปยังเพื่อนบ้านฉากที่อยู่ใน Grid



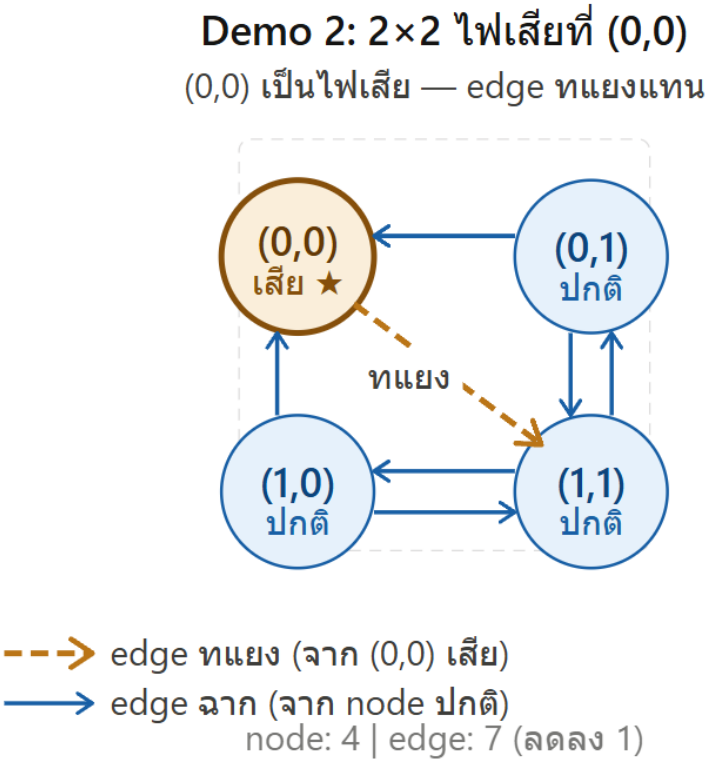
ภาพที่ 2-1: กราฟ directed graph ของ Demo 1 — node วงกลม = หลอดไฟ, ลูกศร = edge แสดงทิศทาง toggle

Node	ประเภท	Edge ขาออก
(0,0)	ปกติ	(0,1), (1,0) — เพื่อนบ้านขวาและล่าง
(0,1)	ปกติ	(0,0), (1,1) — เพื่อนบ้านซ้ายและล่าง
(1,0)	ปกติ	(0,0), (1,1) — เพื่อนบ้านบนและขวา
(1,1)	ปกติ	(1,0), (0,1) — เพื่อนบ้านซ้ายและบน

ตารางที่ 2-2: Edge ของกราฟใน Demo 1 — จำนวน node: 4, จำนวน edge: 8

2.1.5 กราฟจริงของ Demo 2 (2×2 มีไฟเสียที่ (0,0), initial = 1111)

เมื่อเรียก setBroken(0,0) node (0,0) จะเปลี่ยนสถานะเป็น broken โปรแกรมจะลบ edge ขาออกเดิมของ (0,0) และสร้าง edge แนวทแยงใหม่แทน



ภาพที่ 2-2: กราฟ directed graph ของ Demo 2 — (0,0) สีส้มคือไฟเสีย, เส้นประสีส้ม = edge ทแยง

Node	ประเภท	Edge ขาออก
(0,0) ★	เสีย	(1,1) — เพื่อนบ้านทแยงเดียวที่อยู่ใน Grid (มุมบนซ้าย จึงมีทแยงเดียว)
(0,1)	ปกติ	(0,0), (1,1) — ไม่เปลี่ยนแปลง (0,1) เป็นปกติ
(1,0)	ปกติ	(0,0), (1,1) — ไม่เปลี่ยนแปลง
(1,1)	ปกติ	(1,0), (0,1) — ไม่เปลี่ยนแปลง

ตารางที่ 2-3: Edge ของกราฟใน Demo 2 — จำนวน node: 4, จำนวน edge: 7 (ลดลง 1 เพราะ (0,0) ที่มุมมีทแยงเดียว)

2.2 โครงสร้างข้อมูลอื่น ๆ

ชื่อ object	คลาส / Method	ประเภท	หน้าที่และเหตุผลที่ใช้
node	lightboard (field)	light[][]	เก็บ object light ทุกดวงแบบ 2D array เพื่อเข้าถึงหลอดไฟตามพิกัด (row,col) ใน O(1) ขนาดคงที่จึงเหมาะกว่า List
queue	solveBFS()	Queue<BoardState> (ArrayDeque)	เก็บ BoardState ที่รอขยาย เรียงแบบ FIFO ทำให้ BFS ค้นพบคำตอบที่ใช้จำนวน move น้อยที่สุดเสมอ
visited	solveBFS()	Set<String> (HashSet)	เก็บ bits string ของทุก state ที่เคย visit แล้ว ป้องกัน BFS วนซ้ำ state เดิม ค้นหาใน O(1)
history	BoardState (field)	List<String> (ArrayList)	เก็บลำดับ move ในรูป "(row,col)" สำหรับ replay ผลลัพธ์ ใช้ ArrayList เพราะต้องการ add ท้ายและวนซ้ำ
edgesToRemove	setBroken()	Set<DefaultEdge> (HashSet)	เก็บ edge ขาออกเดิมของไฟเสีย เพื่อลบทิ้งก่อนสร้าง edge ทแยงใหม่ หลีกเลี่ยงการแก้ collection ระหว่างวนซ้ำ

ตารางที่ 2-4: โครงสร้างข้อมูลอื่นๆ ในโปรแกรม

3. อัลกอริทึมกราฟ BFS ในการดับไฟ

3.1 อัลกอริทึมที่ใช้: BFS (Breadth-First Search)

โปรแกรมใช้ BFS ค้นหาลำดับการ toggle หลอดไฟที่ทำให้ไฟดับทั้งหมด โดยมีเหตุผลที่เลือก BFS ดังนี้

- BFS ค้นหาแบบ level-by-level จาก state เริ่มต้น state แรกที่ไปถึง all-zeros state จะเป็นคำตอบที่ใช้จำนวน move น้อยที่สุดเสมอ ตรงตามข้อกำหนด extra point ของ project
- DFS ไม่เหมาะ เพราะอาจพบคำตอบที่ใช้ move มากกว่าจำเป็น
- Dijkstra หรือ A* ไม่จำเป็น เพราะทุก move มี cost เท่ากัน (= 1)

3.2 ขั้นตอนการค้นหา (Forwarding Steps)

3.2.1 จุดเริ่มต้นและโครงสร้าง BFS

BFS ทำงานใน method solveBFS() โดยมีขั้นตอนดังนี้

```java
    public int solveBFS(String initialBits) {
        Queue<BoardState> queue = new ArrayDeque<>();
        Set<String> visited = new HashSet<>();

        StringBuilder targetBuilder = new StringBuilder();
        for (int i = 0; i < board * board; i++) {
            targetBuilder.append("0");
        }
        String targetBits = targetBuilder.toString();

        queue.add(new BoardState(initialBits, new ArrayList<>()));
        visited.add(initialBits);

        System.out.println("\nStart searching for solution (BFS)...");
```

- สร้าง targetBits เป็น string "000...0" ขนาด N×N
- ใส่ initial state ลงใน Queue และ visited
- วนซ้ำจนกว่า Queue จะว่างหรือพบ target

3.2.2 การจำลองการ toggle ผ่านกราฟ

ทุกครั้งที่ BFS ทดลองกด node (r,c) โปรแกรมจะเรียก simulatePress() ซึ่งดึง edge ขาออกจากกราฟเพื่อรู้ว่า node ใดบ้างต้องถูก toggle ตาม

```java
    private String simulatePress(String currentBits, int r, int c) {
        char[] bits = currentBits.toCharArray();
        toggleChar(bits, r, c);
        light source = node[r][c];
        for (DefaultEdge e : overboard.outgoingEdgesOf(source)) {
            light neighbor = overboard.getEdgeTarget(e);
            toggleChar(bits, neighbor.getRow(), neighbor.getCol());
        }
        return new String(bits);
    }
```

การเชื่อมต่อระหว่าง BFS และกราฟทำให้ logic การ toggle ถูกต้องอัตโนมัติทั้งไฟปกติ (ฉาก) และไฟเสีย (ทแยง) โดยไม่ต้อง hardcode ทิศทาง

3.2.3 การ replay ผลลัพธ์และ resetNodeState

เมื่อ BFS พบ target โปรแกรมจะ reset node[][] ให้ตรงกับ initialBits ก่อน จากนั้น replay ทีละ move บน node[][] จริงเพื่อแสดงผล

```java
    private void resetNodeState(String bits) {
        for (int r = 0; r < board; r++)
            for (int c = 0; c < board; c++) {
                boolean shouldBeOn = bits.charAt(r * board + c) == '1';
                if (node[r][c].checkon() != shouldBeOn)
                    node[r][c].toggle();
            }
    }
```

3.3 กรณีไม่มีคำตอบ (No Solution)

เมื่อ BFS สำรวจ state ทุกตัวที่ reachable จาก initial state แล้ว (Queue ว่างลง) แต่ยังไม่พบ all-zeros state โปรแกรมจะแสดง No solution!! ทันที กรณีนี้เกิดขึ้นเมื่อ initial state อยู่ใน connected component ของ state space ที่ไม่มี all-zeros state รวมอยู่ด้วย เพื่อพิสูจน์ข้อนี้ เราจะเปรียบเทียบกราฟและ state space ของ Demo 3, 4 และ 5 ซึ่งใช้ initial state เดียวกัน (000101010) แต่ตำแหน่งไฟเสียต่างกัน

3.3.1 กราฟของ Demo 3, 4 และ 5 (initial state = 000101010)

ตารางต่อไปนี้แสดง edge ขาออกของทุก node ใน Demo 3 (ไม่มีไฟเสีย), Demo 4 (ไฟเสียที่ (2,2)) และ Demo 5 (ไฟเสียที่ (0,1)) ค่าที่ต่างกันจาก Demo 3 จะถูกเน้นไว้

Node	ประเภท	Demo 3 (ไม่มีไฟเสีย)	Demo 4 (เสียที่ (2,2))	Demo 5 (เสียที่ (0,1))
(0,0)	ปกติ	(1,0), (0,1)	(1,0), (0,1)	(1,0), (0,1)
(0,1) ★	ปกติ / เสีย	(1,1), (0,0), (0,2)	(1,1), (0,0), (0,2)	**(1,0), (1,2) [**ทแยง**]**
(0,2)	ปกติ	(1,2), (0,1)	(1,2), (0,1)	(1,2), (0,1)
(1,0)	ปกติ	(0,0), (2,0), (1,1)	(0,0), (2,0), (1,1)	(0,0), (2,0), (1,1)
(1,1)	ปกติ	(0,1), (2,1), (1,0), (1,2)	(0,1), (2,1), (1,0), (1,2)	(0,1), (2,1), (1,0), (1,2)
(1,2)	ปกติ	(0,2), (2,2), (1,1)	(0,2), (2,2), (1,1)	(0,2), (2,2), (1,1)
(2,0)	ปกติ	(1,0), (2,1)	(1,0), (2,1)	(1,0), (2,1)
(2,1)	ปกติ	(1,1), (2,0), (2,2)	(1,1), (2,0), (2,2)	(1,1), (2,0), (2,2)
(2,2) ★	ปกติ / เสีย	(1,2), (2,1)	**(1,1) [**ทแยง**]**	(1,2), (2,1)

ตารางที่ 3-1: เปรียบเทียบ edge ขาออกของแต่ละ node ใน Demo 3, 4 และ 5 (★ = node ที่ต่างกัน)

3.3.2 ผลการเปรียบเทียบ State Space

จากโครงสร้าง edge ในตารางที่ 3-1 สามารถคำนวณได้ว่า เมื่อกด node ทุกชุดที่เป็นไปได้จาก initial state “000101010” state ใดบ้างที่ BFS สามารถ reach ได้ ผลลัพธ์ได้ดังนี้

Demo	ไฟเสียที่	Reachable States	มี “000000000” หรือไม่
Demo 3	ไม่มี	512 states (= 2^9)	มีคำตอบ
Demo 4	(2,2)	512 states (= 2^9)	มีคำตอบ
Demo 5	**(0,1)**	**256 states (= 2^8)**	ไม่มีคำตอบ

ตารางที่ 3-2: ผลการสำรวจ state space จาก initial state “000101010” ของ Demo 3, 4 และ 5

3.3.3 สาเหตุที่ Demo 5 ไม่มีคำตอบ

ข้อแตกต่างระหว่าง Demo 3/4 กับ Demo 5 อยู่ที่ edge ของ node (0,1) ซึ่งเป็นไฟเสียใน Demo 5 ดังนี้

Demo 3/4 — (0,1) ปกติ: กด (0,1) จะ toggle ตัวเอง และ (1,1), (0,0), (0,2) รวม 4 หลอด ทำให้สามารถเปลี่ยนค่า bit 0, 1, 2 (แถวบน) และ bit 4 (กลาง) พร้อมกันได้

Demo 5 — (0,1) เสีย: กด (0,1) จะ toggle ตัวเอง และ (1,0), (1,2) เท่านั้น — ไม่ toggle (0,0) และ (0,2) ซึ่งเป็นเพื่อนบ้านฉากปกติ แต่ไปเพิ่ม toggle (1,0) และ (1,2) แทน

ผลกระทบต่อ state space: เมื่อ node (0,1) เปลี่ยน edge ไป การ toggle ทุกชุดที่เป็นไปได้ไม่สามารถสร้าง state “000000000” ได้จาก “000101010” อีกต่อไป state space ของ Demo 5 มีขนาดเพียง 256 states (ครึ่งหนึ่งของ Demo 3/4 ที่มี 512 states) และ “000000000” อยู่ใน connected component อีก component หนึ่งของ state space ซึ่งเข้าไม่ถึงจาก initial state “000101010”

สังเกตว่า state ทั้ง 256 ตัวใน Demo 5 ล้วนมีค่า bit ตำแหน่ง 0, 2 (หลอด (0,0) และ (0,2)) เป็น 0 เสมอ หรือมีแบบแผนเฉพาะที่ไม่รวม “000000000” ในทางตรงกันข้าม Demo 3 และ Demo 4 มี state space ที่ครอบคลุม 2^9 = 512 states ทั้งหมด ซึ่งรวม “000000000” ไว้ด้วย BFS จึงพบคำตอบได้

```
            System.out.println("No solution!!");
            return -1;
```

```
Enter grid size N (N x N, min 2):3
Enter initial state (9 bits, left to right, e.g. 100100100):000101010
      | col 0| col 1| col 2|
row 0 |  0   |  0   |  0   |
row 1 |  1   |  0   |  1   |
row 2 |  0   |  1   |  0   |
Set broken light (Y/N)?y
Enter row of broken light (0-2) =0
Enter col of broken light (0-2) =1
      | col 0| col 1| col 2|
row 0 |  0   |  0x  |  0   |
row 1 |  1   |  0   |  1   |
row 2 |  0   |  1   |  0   |

Start searching for solution (BFS)...
No solution!!
```

4. การสาธิตการทำงานของอัลกอริทึม (Demonstration)

หมายเหตุ: BFS สำรวจ state ทุก node (r,c) ตามลำดับ row 0 ถึง N-1, col 0 ถึง N-1 แต่ละ step แสดงสถานะ bit string และภาพกระดาน visited set ป้องกันการวน state ซ้ำ

4.1 Demo 1: Grid 2×2, ไม่มีไฟเสีย, initial state = 1111

สถานะเริ่มต้น: ไฟทุกดวงติด กดปุ่มใดๆ ก็จะ toggle ตัวเองและเพื่อนบ้านฉาก BFS จะค้นหาลำดับที่สั้นที่สุดเพื่อให้ทุกดวงดับ

<u>Step 0</u> — Initial State

Board: 1111 (ไฟติดทุกดวง)

```
        | col 0| col 1|
row 0   |  1   |  1   |
row 1   |  1   |  1   |
```

Visited set: { "1111" }

Queue: [("1111", history=[])]

BFS เริ่มต้น — ยังไม่เจอ target ("0000")

<u>Step 1</u> — BFS ดึง state "1111" จาก Queue, ลอง toggle (0,0)

เมื่อกด (0,0): toggle (0,0) ตัวเอง + edge ขาออก ⟶ (0,1) และ (1,0)

1111 ⟶ toggle (0,0),(0,1),(1,0) ⟶ 0001

"0001" ยังไม่เคย visit ⟶ เพิ่มเข้า Queue พร้อม history=["(0,0)"]

<u>Step 2</u> — ลอง toggle (0,1) จาก state "1111"

เมื่อกด (0,1): toggle (0,1) ตัวเอง + (0,0),(1,1)

1111 ⟶ toggle (0,1),(0,0),(1,1) ⟶ 0110

"0110" ยังไม่เคย visit ⟶ เพิ่มเข้า Queue

<u>Step 3</u> — ลอง toggle (1,0) จาก state "1111"

เมื่อกด (1,0): toggle (1,0) ตัวเอง + (0,0),(1,1)

1111 ⟶ toggle (1,0),(0,0),(1,1) ⟶ 0110

"0110" เคย visit แล้ว ⟶ ข้าม

Step 4 — ลอง toggle (1,1) จาก state "1111"

เมื่อกด (1,1): toggle (1,1) ตัวเอง + (1,0),(0,1)

1111 ⟶ toggle (1,1),(1,0),(0,1) ⟶ 1000

"1000" ยังไม่เคย visit ⟶ เพิ่มเข้า Queue

BFS ขยายต่อใน Level 2 — ดึง state ถัดไปจาก Queue...

(ข้ามการแสดงรายละเอียดทุก state — BFS ทำงานอัตโนมัติจนพบคำตอบ)

BFS พบคำตอบ — 4 moves

Move	กด (row, col)	State in bits	กระดาน
เริ่มต้น	—	1111	1 1 / 1 1
1	row 0, col 0	0001	0 0 / 0 1
2	row 0, col 1	1100	1 1 / 0 0
3	row 1, col 0	0111	0 1 / 1 1
4	row 1, col 1	0000	0 0 / 0 0

ตารางที่ 4-1: ลำดับ move ของ Demo 1 (BFS พบคำตอบที่ใช้ 4 moves ซึ่งเป็นจำนวนน้อยที่สุด)

รายละเอียดการ replay ทีละ move:

```
>> Move 1 : toggle row 0, col 0

กด (0,0): toggle (0,0) +(0,1) +(1,0)

   State in bits = 0001

ผล: เฉพาะ (1,1) ยังติดอยู่


>> Move 2 : toggle row 0, col 1

กด (0,1): toggle (0,1) +(0,0) +(1,1)

   State in bits = 1100

ผล: (0,0) และ (0,1) ติด, แถวล่างดับ
```

>> Move 3 : toggle row 1, col 0

กด (1,0): toggle (1,0) +(0,0) +(1,1)

State in bits = 0111

ผล: (0,1), (1,0), (1,1) ติด

>> Move 4 : toggle row 1, col 1

กด (1,1): toggle (1,1) +(1,0) +(0,1)

State in bits = 0000

ผล: ไฟดับทุกดวง

4.2 Demo 2: Grid 2×2, มีไฟเสียที่ (0,0), initial state = 1111

สถานะเริ่มต้น: ไฟทุกดวงติด มีไฟเสียที่ (0,0) การกด (0,0) จะ toggle เฉพาะ (1,1) (ทแยงเดียวใน Grid) แทนที่จะเป็น (0,1) และ (1,0) แบบปกติ

Step 0 — Initial State (หลัง setBroken)

Board: 1111 (มีไฟเสียที่ (0,0))

```
        | col 0| col 1|
row 0   |  1x  |  1   |
row 1   |  1   |  1   |
```

กราฟเปลี่ยนแปลง:

- (0,0) ⟶ edge ขาออก: (1,1) เท่านั้น
- (0,1) ⟶ edge ขาออก: (0,0),(1,1) ตามปกติ
- (1,0) ⟶ edge ขาออก: (0,0),(1,1) ตามปกติ
- (1,1) ⟶ edge ขาออก: (1,0),(0,1) ตามปกติ

Step 1 — BFS ดึง state "1111", ลอง toggle (0,0) [ไฟเสีย]

เมื่อกด (0,0) ซึ่งเป็นไฟเสีย: toggle (0,0) ตัวเอง + edge ทแยง ⟶ (1,1) เท่านั้น

1111 ⟶ toggle (0,0),(1,1) ⟶ 0110

"0110" ยังไม่เคย visit ⟶ เพิ่มเข้า Queue พร้อม history=["(0,0)"]

Step 2 — ลอง toggle (0,1) จาก state "1111"

เมื่อกด (0,1): toggle (0,1) ตัวเอง + (0,0),(1,1)

1111 ⟶ toggle (0,1),(0,0),(1,1) ⟶ 0110

"0110" เคย visit แล้ว ⟶ ข้าม

Step 3 — ลอง toggle (1,0) จาก state "1111"

เมื่อกด (1,0): toggle (1,0) ตัวเอง + (0,0),(1,1)

1111 ⟶ toggle (1,0),(0,0),(1,1) ⟶ 0110

"0110" เคย visit แล้ว ⟶ ข้าม

Step 4 — ลอง toggle (1,1) จาก state "1111"

เมื่อกด (1,1): toggle (1,1) ตัวเอง + (1,0),(0,1)

1111 ⟶ toggle (1,1),(1,0),(0,1) ⟶ 1000

"1000" ยังไม่เคย visit ⟶ เพิ่มเข้า Queue

BFS ขยายต่อ...

(BFS ทำงานต่อจนพบคำตอบ — ได้ 3 moves ซึ่งต่างจาก Demo 1 เพราะกราฟต่างกัน)

BFS พบคำตอบ — 3 moves

Move	กด (row, col)	State in bits	กระดาน (x = ไฟเสีย)
เริ่มต้น	—	1111	1x 1 / 1 1
1	row 0, col 0 ★	0110	0x 1 / 1 0
2	row 0, col 1	1011	1x 0 / 1 1
3	row 1, col 0	0000	0x 0 / 0 0

ตารางที่ 4-2: ลำดับ move ของ Demo 2 (BFS พบคำตอบที่ใช้ 3 moves — น้อยกว่า Demo 1 เพราะกราฟต่างกัน)

รายละเอียดการ replay ทีละ move:

>> Move 1 : toggle row 0, col 0 [★ ไฟเสีย — toggle แบบทแยง]

กด (0,0): toggle ตัวเอง + edge ทแยง ⟶ (1,1) เท่านั้น

State in bits = 0110

ผล: (0,0) ดับ, (1,1) ดับ, (0,1) และ (1,0) ยังติด

```
       | col 0| col 1|
row 0  |  0x  |  1   |
row 1  |  1   |  0   |
```

>> Move 2 : toggle row 0, col 1 [ไฟปกติ — toggle แบบฉาก]

กด (0,1): toggle ตัวเอง + (0,0),(1,1)

State in bits = 1011

ผล: (0,0) ติด, (0,1) ดับ, (1,1) ติด

```
       | col 0| col 1|
row 0  |  1x  |  0   |
row 1  |  1   |  1   |
```

>> Move 3 : toggle row 1, col 0 [ไฟปกติ — toggle แบบฉาก]

กด (1,0): toggle ตัวเอง + (0,0),(1,1)

State in bits = 0000

ผล: ไฟดับทุกดวง

```
       | col 0| col 1|
row 0  |  0x  |  0   |
row 1  |  0   |  0   |
```

เปรียบเทียบ Demo 1 และ Demo 2:

ลักษณะ	Demo 1 (ไม่มีไฟเสีย)	Demo 2 (ไฟเสีย (0,0))
จำนวน edge ในกราฟ	8	7 (ลดลง 1)
Edge ขาออกของ (0,0)	(0,1), (1,0) — ฉาก	(1,1) เท่านั้น — ทแยง
จำนวน move ที่ใช้	4 moves	3 moves
คำตอบ	(0,0)⟶(0,1)⟶(1,0)⟶(1,1)	(0,0)⟶(0,1)⟶(1,0)

ตารางที่ 4-3: เปรียบเทียบ Demo 1 และ Demo 2

5. ข้อจำกัดของโปรแกรม (Program Limitation)

5.1 ข้อจำกัดด้านหน่วยความจำและเวลา (BFS State Space)

BFS ต้องเก็บทุก state ที่เคย visit ใน HashSet (visited) และ state ที่รอขยายใน Queue จำนวน state ทั้งหมดของกระดาน N×N คือ 2^(N×N) state ดังนี้

ขนาด Grid	N×N	จำนวน State สูงสุด	ประมาณ
2×2	4	2^4 = 16	เร็วมาก
3×3	9	2^9 = 512	เร็วมาก
4×4	16	2^16 = 65,536	เร็ว
5×5	25	2^25 ≈ 33 ล้าน	ช้ามาก / อาจ OOM

ตารางที่ 5-1: จำนวน state สูงสุดตามขนาด Grid — โปรแกรมเหมาะสำหรับ Grid ไม่เกิน 4×4

- ข้อจำกัดที่แท้จริงคือ memory และเวลา ไม่ใช่ขนาด Grid ที่ป้อนได้ (โปรแกรมรับ N ใดก็ได้ $\geq$ 2 แต่อาจ freeze หรือ OutOfMemoryError สำหรับ N $\geq$ 5)

5.2 ไฟเสียรองรับได้สูงสุด 1 ดวง

โปรแกรมรองรับไฟเสียได้เพียง 1 ดวงต่อเกม ตามข้อกำหนดของ project หาก Grid จำเป็นต้องมีไฟเสียมากกว่า 1 ดวง ต้องแก้ไข method setBroken() และ createEdgesForNode() ให้รองรับหลาย node ที่เป็น broken พร้อมกัน

5.3 การ copy history ทุก state ทำให้ใช้ memory มากกว่าจำเป็น

ใน solveBFS() ทุกครั้งที่ BFS ขยาย state ใหม่ โปรแกรมจะทำ new ArrayList(current.history) เพื่อ copy รายการ move ทั้งหมดไปเก็บใน BoardState ใหม่ทุก state ที่สร้างขึ้น ทำให้ memory ที่ใช้จริงเป็น O(S × D) โดย S คือจำนวน reachable states และ D คือความลึกสูงสุดของ BFS แทนที่จะเป็น O(S) หากใช้ parent pointer แทน ข้อจำกัดนี้ทำให้โปรแกรมใช้ memory มากกว่าจำเป็นโดยเฉพาะเมื่อ Grid มีขนาดใหญ่และมีจำนวน move สูง

บรรณานุกรม

JGraphT. (n.d.). JGraphT – A Free Java Graph Library. https://jgrapht.org/javadoc/

Wikipedia. (n.d.). Lights Out (game). https://en.wikipedia.org/wiki/Lights_Out_(game)

Oracle. (n.d.). ArrayDeque (Java SE 17). https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/ArrayDeque.html

Oracle. (n.d.). HashSet (Java SE 17). https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/HashSet.html

GeeksforGeeks. (n.d.). BFS for a Graph. https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/

Oracle. (n.d.). DefaultDirectedGraph – JGraphT 1.5.1. https://jgrapht.org/javadoc/org/jgrapht/graph/DefaultDirectedGraph.html

แบบฟอร์มรับรองการใช้งาน AI

วัตถุประสงค์ของเอกสารฉบับนี้คือ เพื่อแสดงความโปร่งใสและจริยธรรมในการใช้งาน Generative AI ในการทำโครงงาน

โครงงานกลุ่มเรื่อง: Light Out Puzzle

คณะผู้จัดทำขอรับรองว่ามีการใช้เครื่องมือ Claude (Anthropic) เพื่อ:

ทำเครื่องหมายหน้าหัวข้อที่มีการใช้งาน

☐ 1. ระดมความคิด (brainstorm) และวางโครงร่างเนื้อหาสำหรับการวิจัยเบื้องต้น

☐ 2. ระบุและค้นหาแหล่งข้อมูลที่เกี่ยวข้อง

☒ 3. สรุปเนื้อหาจากเอกสาร (technical document) โดยข้อความที่ถูกสร้างขึ้นนั้น:

- ☐ ใช้เพื่อช่วยในการทำความเข้าใจแนวคิดเท่านั้น แต่ไม่ได้รวมอยู่ในรายงาน
- ☒ ใส่ไว้ในรายงาน ส่วนที่ 3 และ 4

☒ 4. สร้างหรือเสนอแนะแนวทางในการแก้ไขปัญหา ดังนี้:

- *การออกแบบโครงสร้างกราฟ และการค้นหาเส้นทางเพื่อดับไฟ*

☒ 5. สร้างโค้ดเพื่อใช้ในการแก้ปัญหา ดังนี้:

- *solveBFS(), simulatePress(), resetNodeState() ในคลาส lightboard*

☒ 6. อธิบายหรือปรับเปลี่ยนสำนวน (rephrase) แนวคิด หรืออธิบายโค้ดหรืออัลกอริทึมเป็นคำพูด หรือสร้างตัวอย่างเพื่อขยายความแนวคิด/โค้ด/อัลกอริทึม โดยข้อความที่ถูกสร้างขึ้นนั้น:

- ☐ ใช้เพื่อช่วยในการทำความเข้าใจแนวคิด/โค้ดเท่านั้น แต่ไม่ได้รวมอยู่ในรายงาน
- ☒ ใส่ไว้ในรายงาน หน้า/ส่วนที่ ส่วนที่ 3.2 และ 4

☒ 7. วิเคราะห์หรือเปรียบเทียบระหว่างแนวทางการแก้ปัญหาหรือวิธีการ โดยข้อความที่ถูกสร้างขึ้นนั้น

- ☐ ใช้เพื่อช่วยในการทำความเข้าใจแนวคิดเท่านั้น แต่ไม่ได้รวมอยู่ในรายงาน
- ☒ ใส่ไว้ในรายงาน ส่วนที่ 3.1

☒ 8. สร้างตาราง แผนภูมิ หรือแผนผัง จากข้อความหรือข้อมูล ดังนี้:

- *ตารางที่ 2-1 ถึง 5-1, ตารางที่ 3-1 และ 3-2 (เปรียบเทียบ state space Demo 3-5)*

☐ 9. ตรวจสอบไวยากรณ์ ตรวจทาน หรือปรับปรุงงานเขียน

☒ 10. สร้างการอ้างอิงในเนื้อความ (citations) และรายการอ้างอิง/บรรณานุกรม (references)

☒ 11. อื่นๆ (ช่วยตรวจสอบความถูกต้องของตัวอย่างผลลัพธ์ BFS ในส่วนที่ 4)

คณะผู้จัดทำขอยืนยันว่า เนื้อหาทั้งหมดที่สร้างโดย AI ที่นำมาใช้ในโครงงานนี้ ได้ผ่านการตรวจสอบเพื่อยืนยันความถูกต้องและความเข้าใจโดยคณะผู้จัดทำเป็นที่เรียบร้อยแล้ว

สมาชิกกลุ่ม:

นางสาวกรชนก ภู่ตระกูล 6713115

นางสาวณัฏฐา ลิ้มคุณธรรมโม 6713117

นาย จักริน เริ่มตั้งสกุล 6713221